# BAP: Budgeted Activation Propagation for Sparse Graph-Based Reasoning

Replacing Token Generation with Structured Thought Traversal

**Chandan S H**

Karnataka, India

2026 | Apache License 2.0

**Abstract**

We present BAP (Budgeted Activation Propagation), a graph-based reasoning engine that replaces token-level chain-of-thought generation with structured traversal over a sparse semantic thought graph. BAP models reasoning as energy-budget-controlled activation propagation across atomic knowledge primitives — facts, rules, functions, and constraints — retrieved by SHSRS (Spherical Hierarchical Semantic Retrieval System), a two-stage hierarchical index that enables sub-linear search over large reasoning corpora. On benchmarks across three domains (fibonacci reasoning, arithmetic, multihop facts), BAP achieves 100% hit rate with an average Token Reduction Ratio (TRR) of 0.021 compared to chain-of-thought reasoning with the same underlying language model, corresponding to a 47.5x reduction in reasoning tokens. We further introduce dynamic thought space expansion via LLM-governed node generation, outcome-weighted reinforcement learning over graph structure, multi-domain routing, and drift and stagnation detection as control mechanisms. The architecture is designed as a kernel — stable, interpretable, and extensible — with a plug-and-play LLM interface for semantic generation. BAP is released under the Apache 2.0 license.

## 1. Introduction

Large language models solve reasoning problems by generating chains of thought as token sequences. While effective, this approach is expensive: a single reasoning query requires 100–500 tokens of generated text, most of which is structural scaffolding rather than domain knowledge. The reasoning content itself — the core inferential steps — is a small fraction of total generation cost.

We ask a different question: can reasoning be executed without generating tokens at all? If the relevant knowledge primitives are already encoded in a structured graph, reasoning becomes a traversal problem — finding a path through the graph that answers the query — rather than a generation problem. Traversal over a 200-node graph costs 5 steps. Token generation costs 315 tokens. The difference is an order of magnitude.

BAP (Budgeted Activation Propagation) realizes this idea. It maintains a ThoughtSpace — a domain-specific graph of atomic reasoning primitives — and traverses it using an activation equation that balances semantic similarity, node quality (gate), temporal decay, energy budget, and path novelty. The LLM is not eliminated; it is repositioned. Instead of generating the full reasoning chain, it generates single atomic nodes on-demand when retrieval fails or when a concept is absent

from the graph.

This paper makes the following contributions:

1. BAP: a graph-based reasoning architecture with a formal activation equation replacing token-level CoT.
2. SHSRS: a hierarchical two-stage sparse retrieval system enabling sub-linear search over thought graphs.
3. Dynamic thought space expansion: LLM-governed node generation with outcome-weighted reinforcement.
4. Multi-domain routing: automatic query-to-domain assignment via best-match SHSRS search.
5. TRR (Token Reduction Ratio): a metric for comparing token-based and graph-based reasoning efficiency.
6. A complete open-source kernel implementation validated at 100% hit rate and 47.5x token reduction.

## 2. Related Work

Chain-of-thought prompting [Wei et al., 2022] demonstrated that LLMs reason better when intermediate steps are generated explicitly. Subsequent work explored self-consistency [Wang et al., 2023], tree-of-thought search [Yao et al., 2023], and graph-of-thought [Besta et al., 2023]. These approaches extend CoT but remain token-generation-centric: reasoning cost scales with output token length.

Retrieval-augmented generation (RAG) [Lewis et al., 2020] introduced retrieval as a complement to generation, but retrieval surfaces documents rather than reasoning steps. Knowledge graph question answering [Sun et al., 2019] traverses structured graphs but requires pre-existing relational databases and does not model activation dynamics or budget constraints. Neural process networks and differentiable reasoning systems [Andreas et al., 2016] perform structured inference but require task-specific training and lack persistent memory.

BAP differs from all of the above in three ways: (1) reasoning is traversal, not generation; (2) the graph is self-expanding via controlled LLM injection; and (3) the system is governed by an energy budget that enforces computational constraints without task-specific training.

## 3. Architecture

### 3.1 ThoughtSpace and ThoughtNodes

A ThoughtSpace is a domain-specific collection of atomic reasoning primitives called ThoughtNodes. Each node encodes one indivisible piece of knowledge:

| Field | Type | Description |
|---|---|---|
| text | str | The atomic knowledge primitive (under 25 words) |
| type | str | fact \| rule \| function \| constraint \| macro |
| embed | List[float] | Sentence embedding (all-MiniLM-L6-v2, dim=384) |

| | | |
|---|---|---|
| strength | float | Learned quality signal, range [0.05, 0.95] |
| age | float | Decay accumulator, incremented per session |
| harm | float | Cosine distance to harm cluster |

*Table 1: ThoughtNode fields.*

### 3.2 The Activation Equation

At each traversal step, BAP scores candidate nodes using the activation equation:

```
A(j) = A_source * sim(i,j) * gate(j) * decay(j) * energy(t) * novelty(j)
```

where each term is defined as:

| Term | Formula | Role |
|---|---|---|
| **`sim(i,j)`** | cosine(embed_i, embed_j) | Semantic relevance to current node |
| **`gate(j)`** | sigmoid(k * (strength_j - harm_j)) | Node quality filter, k=3.0 |
| **`decay(j)`** | exp(-age_j * lambda) | Temporal relevance, lambda=0.01 |
| **`energy(t)`** | log(1 + budget_remaining) | Budget pressure on traversal |
| **`novelty(j)`** | 1 - max cosine(embed_j, path) | Suppresses semantic repetition |

*Table 2: Activation equation terms.*

Traversal proceeds for up to max_steps iterations or until the budget is exhausted (GLOBAL_BUDGET=1.0). While budget > 50%, the top-2 candidates are selected (explore mode); below 50%, only the top-1 candidate is selected (exploit mode). Nodes are hard-pruned if their harm score exceeds 0.5 or if their cosine similarity to any path node falls below -0.15 (contradiction gate). Nodes below MIN_ACTIVATION=0.008 are pruned.

### 3.3 SHSRS: Spherical Hierarchical Semantic Retrieval System

Naive nearest-neighbor search over a growing ThoughtSpace is O(N) per step. SHSRS reduces this to approximately O(sqrt(N)) via two-stage retrieval:

Stage 1 — Centroid search: MiniBatchKMeans clusters node embeddings into K = min(max(N//20, 5), 50) centroids. The query vector is compared to all centroids, and the top-P clusters are selected (P scales with N: 2 for N<60, 6 for N<150, 12 for N<300, 20 for N>=300).

Stage 2 — Node scoring: Within the selected clusters, the top-K nodes by cosine similarity are returned as candidates (K=10). Activation scoring runs only on these candidates.

Goal attraction is applied at each step: search_vec = 0.70 * current_vec + 0.30 * query_vec, blending the local context with the original query to prevent semantic drift on long paths.

### 3.4 Dynamic Thought Space Expansion

Two generation triggers extend the ThoughtSpace at runtime:

OOV Detection: Before traversal begins, BAP scans query terms against the corpus. Terms with maximum corpus cosine similarity below OOV_THRESHOLD=0.38 (unigrams) or BIGRAM_OOV_THRESHOLD=0.50 (bigrams) are classified as out-of-vocabulary. The LLM backend generates a definitional primitive for the missing concept, which is injected into the

graph before traversal. The query vector is re-embedded as a 0.6/0.4 blend of the original and the definition.

Retrieval Miss: When no candidate exceeds MIN_ACTIVATION after SHSRS search, BAP calls the LLM backend to generate a contextual continuation node from the current path. Generated nodes enter the graph at strength=0.65 and are eligible for reinforcement and retirement.

Prompt injection protection: OOV terms are sanitized before entering prompts — stripped to alphanumeric characters, truncated to 60 chars, and checked against a rejection list of injection keywords.

### 3.5 Outcome-Weighted Reinforcement

After each query, BAP updates node strengths based on outcome and query relevance:

```
delta(j) = REINFORCE_MAX_DELTA * cosine(embed_j, query_vec) [success]
delta(j) = -REINFORCE_MIN_DELTA * cosine(embed_j, query_vec) [failure]
```

Nodes irrelevant to the query receive near-zero updates regardless of outcome. This prevents hub nodes — those appearing on many paths — from accumulating unearned strength. Strengths are clamped to [0.05, 0.95] after each session and renormalized if more than 20% of nodes approach the ceiling or floor.

### 3.6 Control Mechanisms

BAP includes two path-level control mechanisms that trigger LLM injection without a retrieval miss:

Drift Detection: If the mean pairwise cosine of the last DRIFT_WINDOW=3 path embeddings exceeds DRIFT_THRESHOLD=0.85, the path is circling a semantic cluster. A bridge node is injected immediately (action=drift_bridge). Fires at most once per run.

Stagnation Detection: If mean novelty over the last STAGNATION_WINDOW=4 steps falls below STAGNATION_NOV_FLOOR=0.15 AND node types are repeating, a bridge node is injected (action=stagnation_bridge). Distinct from drift — stagnation detects novelty plateau, drift detects semantic looping.

### 3.7 Multi-Domain Routing

DomainRouter routes queries across multiple registered ThoughtSpaces. For each domain, it runs a k=1 SHSRS search and selects the domain with the highest top-1 cosine score. This is more robust than centroid similarity because large diverse corpora have blurry centroids; the best-matching node in the correct domain always wins cleanly. Cold-start boost (x1.25) is applied to domains with fewer than 100 nodes to prevent large domains from drowning out newly registered ones. A routing margin warning fires when the gap between the top two domains is below 0.15.

### 3.8 Plug-and-Play LLM Interface

BAP decouples reasoning control from semantic generation via an abstract LLMBackend interface with two methods: available() and generate_primitive(prompt). Four implementations are provided: MockBackend (testing), OllamaBackend (local models), OpenAIBackend, and AnthropicBackend. A register_backend() function enables runtime registration of any custom backend. BAP never changes when the backend changes.

## 4. Experimental Results

### 4.1 Benchmark Setup

We evaluate BAP on three domains, each with 200 atomic nodes populated from curated knowledge primitives. Benchmark queries are designed to require 2-6 reasoning steps. Hit rate measures whether the answer concept appears in the reasoning path. TRR is computed as BAP steps / CoT completion tokens using the same model (llama-3.1-8b-instant via Groq API) with a standard step-by-step CoT prompt.

### 4.2 Domain Benchmark Results

| Domain | Nodes | Avg Steps | Avg TRR | Hit Rate | Status |
|---|---|---|---|---|---|
| fibonacci_reasoning | 200 | 5.1 | 0.237 | 100% | PASS |
| arithmetic | 200 | 5.0 | 0.248 | 100% | PASS |
| multihop_facts | 200 | 4.8 | 0.310 | 100% | PASS |
| **Target** | **—** | **—** | **< 0.40** | **> 95%** | **—** |

*Table 3: BAP benchmark results across all three domains.*

### 4.3 True CoT Baseline Comparison

| Domain | Avg BAP Steps | Avg CoT Tokens | True TRR | Token Reduction |
|---|---|---|---|---|
| fibonacci_reasoning | 5.1 | 349.3 | 0.0167 | ~60x |
| arithmetic | 5.0 | 356.4 | 0.0191 | ~52x |
| multihop_facts | 4.8 | 241.2 | 0.0273 | ~37x |
| **GLOBAL** | **5.0** | **315.6** | **0.0210** | **~47.5x** |

*Table 4: True TRR comparison. CoT tokens measured on llama-3.1-8b-instant with chain-of-thought prompting. Estimated TRR (0.237-0.310) was 14-15x too pessimistic because real CoT generates 241-356 tokens vs the assumed 30-80.*

### 4.4 Multi-Domain Routing Accuracy

DomainRouter was evaluated on 7 queries spanning all three domains after corpus expansion to 200 nodes each. Routing accuracy: 7/7 (100%). Notable: 'Who invented the telephone?' and '17 mod 5' — previously failing — both passed after corpus expansion added relevant anchors. Routing scores ranged from 0.675 to 0.781 for correct domain assignments.

### 4.5 Hardening and Failure Policies

13 hardening policies were implemented and verified without benchmark regression. Key policies include: atomic write with backup for nodes.json, file lock for concurrent access, embedding model version check on load, index/nodes sync validation, strength renormalization, cross-session text dedup via hash index, prompt injection sanitization, domain bleed detection, cold-start boost, node retirement, and adaptive scaling of SHSRS parameters with corpus size.

## 5. Macro Reasoning Nodes (Planned Extension)

We describe a planned extension to BAP that enables self-optimizing reasoning through path compilation. Frequently traversed reasoning paths are compiled into macro nodes — single graph

nodes that represent entire multi-step reasoning chains.

A path qualifies for compilation when: frequency >= 3 runs, path length >= 3 nodes, success rate >= 95%, all constituent nodes remain active, and no existing macro covers the same subpath. On compilation, the LLM backend generates a one-sentence summary as the macro text, and the macro's anchor vector is set to the mean of query vectors that triggered the path.

Macro matching uses fuzzy retrieval: if the query embedding is within cosine 0.82 of the macro's anchor vector, the macro enters the candidate pool with a 1.35x activation boost. On a macro hit, the subpath nodes are expanded inline — the reasoning path remains fully interpretable. Nested macros are allowed to depth 2 (macro-of-macro). Macro invalidation runs at the start of each session, retiring macros whose constituent nodes have been retired.

Projected impact: if macro hits reduce avg steps from 5 to 1-2, true TRR would fall from 0.021 to approximately 0.003-0.006, representing a 150-300x token reduction versus CoT. The system becomes faster as it learns — a property absent from token-generation-based reasoning systems.

## 6. Discussion

BAP demonstrates that for structured knowledge domains, reasoning can be executed as graph traversal at a fraction of the token cost of chain-of-thought generation. The 47.5x token reduction is not an artifact of query selection — it reflects a fundamental difference in how the two paradigms represent and execute reasoning: CoT externalizes reasoning as text; BAP internalizes it as graph structure.

The LLM is not eliminated in BAP — it is repositioned. Rather than generating the full reasoning chain, it acts as an on-demand abstraction engine, called only when the graph lacks coverage (OOV or retrieval miss). This governance model means BAP inherits the flexibility of LLMs while retaining deterministic control over traversal, budget, and path interpretability.

Limitations: BAP currently requires domain-specific ThoughtSpace construction. The quality of reasoning paths depends on corpus coverage. Novel cross-domain queries may fall below the router confidence threshold. The true TRR advantage diminishes on highly novel queries where LLM generation dominates traversal.

Future directions include: macro node compilation for self-optimizing paths, parallel traversal (multiple simultaneous path explorations), cross-domain ThoughtSpace merging, and evaluation on standard reasoning benchmarks (GSM8K, HotpotQA, ARC) with automated ThoughtSpace construction.

## 7. Conclusion

We presented BAP, a reasoning kernel that replaces chain-of-thought token generation with budgeted activation propagation over a sparse semantic thought graph. BAP achieves 100% hit rate across three domains with a 47.5x measured token reduction versus the same LLM using chain-of-thought prompting. The system is stable, interpretable, extensible via a plug-and-play LLM interface, and hardened against 13 identified failure modes at scale. We release BAP under the Apache 2.0 license as a foundation for graph-based reasoning research.

## References

[1] Wei, J., et al. (2022). Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. NeurIPS 2022.

[2] Wang, X., et al. (2023). Self-Consistency Improves Chain of Thought Reasoning in Language Models. ICLR 2023.

[3] Yao, S., et al. (2023). Tree of Thoughts: Deliberate Problem Solving with Large Language Models. NeurIPS 2023.

[4] Besta, M., et al. (2023). Graph of Thoughts: Solving Elaborate Problems with Large Language Models. AAAI 2024.

[5] Lewis, P., et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS 2020.

[6] Sun, H., et al. (2019). PullNet: Open Domain Question Answering with Iterative Retrieval on Knowledge Bases and Text. EMNLP 2019.

[7] Andreas, J., et al. (2016). Neural Module Networks. CVPR 2016.

[8] Reimers, N., Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. EMNLP 2019.

[9] Johnson, J., et al. (2019). Billion-scale similarity search with GPUs. IEEE Transactions on Big Data.

---